# Multi UAV Search and Rescue in Crowded Environments

Abirami Elangovan, Aatish Gupta, Vaibhav Parekh

December 2024

## 1 Abstract

UAVs can be useful in search and rescue operations to survey unknown areas and generate maps to be used by rescuers. This project developed a coordinated multi-UAV system for deployment in crowded environments. A multi-goal RRT planner was implemented, which demonstrated asymptotic completeness with an increase in the number of UAVs utilized. This addresses limitations of basic RRT planners. Next Best Viewpoint generation for UAV goal positions was implemented to attempt to improve the information gain during surveillance and map development.

## 2 Introduction

Earthquake sites are filled with hazards that human first responders may be susceptible to: gas leaks, electrical hazards, and sharp debris, to name a few. UAVs can be leveraged to map disaster sites and identify the locations of victims so that first responders can efficiently retrieve them and administer aid. Indoor environments are particularly difficult to explore after an earthquake because of the crowding created by debris.

This project aimed to develop a robust multi-UAV search and rescue system for crowded environments. Lidar sensors mounted on each UAV agent were utilized to build occupancy maps in unknown environments, with each UAV controlled via a multi-goal RRT algorithm. The success of the system was evaluated via development and simulation in MATLAB. By leveraging advanced algorithms for SLAM, path planning, and multi-robot coordination, the project enabled efficient search capabilities in simulated indoor disaster zones.

# 3 Approach

## 3.1 Mathematical Definition of Problem

The states of the UAV, and thus the start and goal states, were defined as $\{x, y, z, \Theta\}$. Only $\Theta$ (heading) of the standard UAV full state was considered to account for the sensor orientation for mapping.

## 3.2 Toolboxes and Models

The UAV and Navigation Toolboxes in MATLAB provided the basis for model and control simulation. The `UAVDubinsConnections` Kinematic Model provided dynamically feasible connected cells for a fixed-wing UAV. For developing the map, 3D Occupancy maps were used. A `validatorOccupancyMap3D` object determined free locations for the UAV.

## 3.3 High-Level Algorithm

### 3.3.1 Occupancy Map and Next Best Viewpoints

When mapping an unknown crowded environment, each UAV would begin at its initial start location (red). From this location, lidar sensor readings developed a new occupancy map, which was a new section of the complete environment. Next Best Viewpoint (NBV) planning (further described in section 3.4) determined the locations in the map (green) with the most potential information gain to best complete a 3D reconstruction. This results in $N_{\text{drones}}$ NBVs, i.e., `GoalPoses`.

(a) Initial NBVs

(b) Updated NBVs

Figure 1: Occupancy Map NBV Following.

### 3.3.2 RRT Planner

With $N_{\text{drones}}$ NBVs generated, each UAV then runs its own multi-goal RRT search, described further in section 3.5, and is assigned a goal.

### 3.3.3 UAV Trajectory Following

Each UAV then follows the solution trajectory for its assigned goal and takes new lidar scans. The lidar scans from each UAV are compiled to create a new occupancy map. The above steps are repeated until the maximum number of high level iterations is completed.

The following sections dive further into the low-level algorithms at each step.

## 3.4 NBV Generation

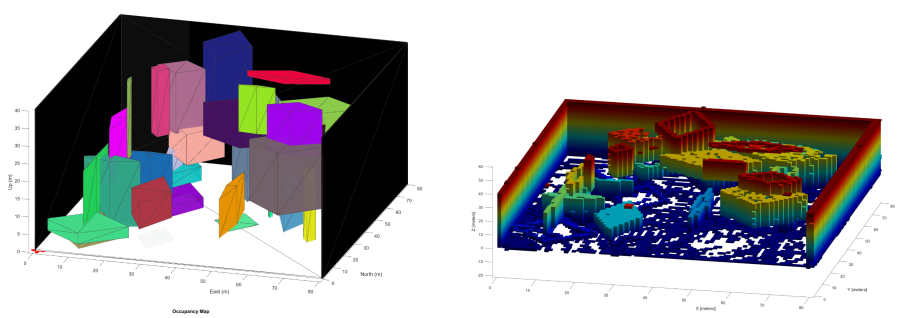
Lidar scans result in an occupancy map - an evenly spaced field of values representing the probability of the presence of an obstacle at that location in the environment. The NBV is defined as the location on the map that would result in the largest information gain during the next iteration of sensor data collection.

1) **Define Thresholds:** The `OccupancyThreshold` (0.4) and `FreeThreshold` (0.7) were defined as the probabilities below and above which a location on the map was confirmed occupied or free.

2) **Develop Binary Occupancy:** `getOccupancy` scanned the entire current map, converting it to a binary 3D matrix indicating obstacle locations.

3) **Explore-Inspect Algorithm:** 100 dynamically feasible, unoccupied locations in the UAV state space were sampled. `simulateLidarSensor` created a prediction of a new binary map if each UAV carried out their scans in one of these new potential locations. The information gain was defined as the sum over this entire map.

4) **Determine NBV:** The $N_{\text{drones}}$ locations with the largest information gains were set as the NBVs. These were thus the $N_{\text{goals}}$ goal poses fed into each UAV multi-goal RRT.




(a) Ground Truth Occupancy Map

(b) Generated Occupancy Map

Figure 2: Comparison between the Ground Truth and Generated Occupancy Maps.

### 3.5 Multi-Goal RRT

The live script creates a `swarm` which stores $N_{\text{drones}}$ instances of the `Drone` class. This class notably contains each UAV's `StartPose`, `GoalPoses`, `planner`, which is an instance of the `plannerMultiUAVMultiGoalRRT` class, as well as variables to store information about the results of that specific UAV's planner - `GoalCosts`, `GoalSolns`, `SolnInfo`, and `AssignedGoal`.

The planner expands on the basic RRT implementation:

1) **Initialize Goal Statuses:** The constructor first creates a $N_{\text{goals}}$ boolean array `ReachedGoals` that tracks whether each goal has been reached, initializing them all to false. `tentativeGoalIds`, an array storing the node IDs corresponding to found goals, is also empty.

2) **Validate Poses and Initialize Tree:** The start and goal poses are confirmed via `validateStartGoal` to be dynamically feasible. A MATLAB `SearchTree` is intialized with the start state.

3) **Sample:** A state is randomly sampled from the unoccupied state space, with a `GoalBias`. Only a goal that has not yet been reached can be sampled, which is tracked by `ReachedGoals`, as previously described.

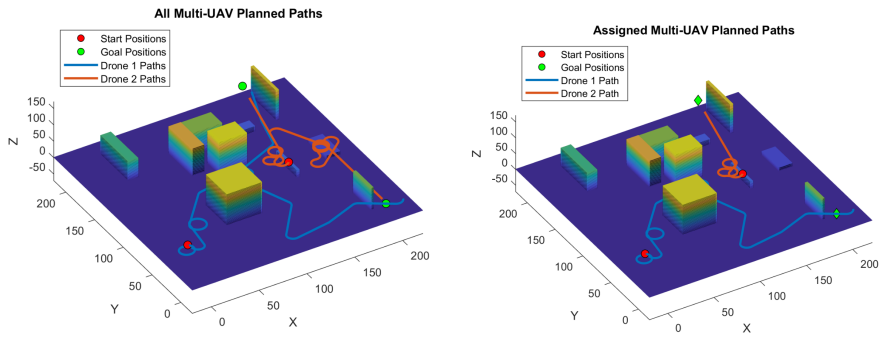
4) **Extend:** Status codes track the progress of the search. Within `extend`, `InProgress` is set and the extension begins, interpolating from the nearest neighbor state to the randomly sampled state up to `MaxConnectionDistance`, potentially setting the status to `MotionInCollision` if trapped. The new node and index is inserted into the tree.

5) **Check Goal Reached:** This new node is then checked against the goal poses. `GoalReachedFcn` defines that if the norm of the vector from the current $\{x, y, z\}$ to any of the goal $\{x, y, z\}$ is within `goalThreshold`, that goal has been reached. If this is a new goal reached, `ReachedGoals` is updated. If all of the goals have been reached, the status is set to `AllGoalsReached`. Otherwise, `GoalReached` is set.

6) **Update Solutions:** The extend function returns with the updated status. If a new goal is reached (`GoalReached`), the new node index is inserted into `tentativeGoalIDs`. Steps 3-6 are repeated until maximum iterations or `AllGoalsReached`.

7) **Post Planning:** For each goal the RRT was able to reach, (stored during step 6), the solution is backtraced, cost is found, and these are all stored. The cost is the length of UAV Dubins Connection path from the start to the goal.

8) **Hungarian Algorithm:** A $N_{\text{drones}}$ x $N_{\text{goals}}$ cost matrix is created, populated with the stored costs for each UAV to reach each feasible goal. If a goal was not reached, this cost was set to a large value to allow solving - 1e6. This cost matrix is then minimized to assign goals to each UAV. If a UAV is unable to reach a specific goal, it is prioritized to be assigned a feasible goal. If none can be reached, the UAV is marked as Idle and stays at its current location until the next occupancy map iteration. Its `AssignedGoal` is updated.




(a) Step 7: All Solutions Found per UAV (b) Step 8: Assigned Solution per UAV by Hungarian Algorithm

Figure 3: Multi-Goal RRT and Goal Assignment Results for N=2.

# 4 Experimental Results

## 4.1 RRT

As expected, solve time increased with the number of maximum iterations. However, increasing maximum iterations beyond 400 showed diminishing returns in completeness. Increasing the goal threshold beyond 5 resulted in solutions ending unreasonably far from the NBV. Thus, these parameters were set and analyzed further.

Hyperparameters: Max Iter: 400, Goal Threshold: 5, Goal Bias: 0.1, Max Connect Dist: 50

Table 1: Drone Planning Time and Cost

| $\mathbf{N_{drones}}$ | **Planning Time** [s] | **Cost** [**Min-Max**] |
|---|---|---|
| 1 | 0.38 | 286 |
| 2 | 1.78 (+1.40) | 225-529 |
| 3 | 2.95 (+1.17) | 333-731 |
| 4* | 2.96 (+0.0024) | 56-876 |
| 5 | 4.73 (+1.77) | 150-1235 |

Overall, it was observed that time scaled with number of UAVs. There is certainly some variability depending on initial poses. For example, case 4 has lower than expected time and cost due to one UAV starting closer to the goal poses. This dependency on initial states means that, as expected of vanilla RRTs, solution completeness for each of the goals is not guaranteed.

However, similar to how increasing iterations guarantees asymptotic completeness, increasing the number of UAVs appears to behave the same. Increasing simply from 1 to 2 UAVs consistently allows for all UAVs combined to reach

two goal states, and this has been observed with greater N as well.

As observed, the paths do appear to decrease in quality with increased N. This makes sense as with a constant value of maximum iterations, less nodes are available to develop each path to the goals. A fix to this could have been to increase iterations with N: N*400. Post-processing smoothing, however, also improves the quality of paths. This process involved searching for distant waypoints, and if no obstacle was found between them, removal of intermediate waypoints.

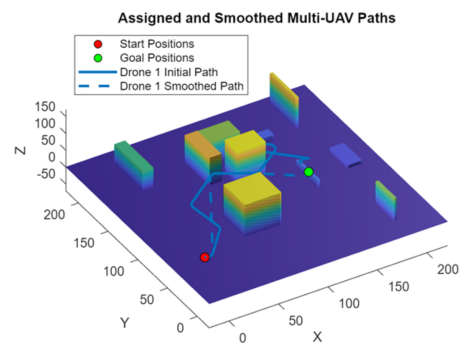



Figure 4: Smoothed Paths

## 4.2 Full Integration and Occupancy Map Generation

https://www.youtube.com/watch?v=nh8o7OpUQHE

The video linked above demonstrates the results of simulating the complete multi-UAV system over three randomly generated occupancy maps.

The first clip for each shows the movements the drone takes from each RRT/NBV generation iteration to the next iteration. This demonstrates the multi-UAV multi-goal RRT planner's capability to manage multiple drones and find dynamically feasible solutions to each of the NBVs over the course of the simulation.

| **Next Best View** | **Random View** |
|---|---|
| 6.5493% | 6.7141% |
| 6.3934% | 6.4827% |
| 6.8743% | 6.9741% |

Table 2: Comparison of Next Best View algorithm and random selection for selecting next drone poses

We tested the NBV algorithm against simply randomly selecting viable points in the map for the drone to travel to. As shown in table 2, we determined that the NBV planner was comparable, but ultimately underperformed compared to random selection.

# 5 Conclusions

This projected succesfully implemented a multi-goal RRT for coordinated multi-UAV surveillance. Time and cost scaled with an increase in the number of drones. However, this provided the benefit of asymptotic completeness of solutions to all goals. This combats a a major drawback of basic RRT implementations. The generation of Next Best Viewpoints was also implemented, but failed to improve the completeness of occupancy map generation for random, unknown environments.

## 5.1 Future Work

Future work includes tackling individual UAV RRT asymptotic completeness via RRT-connect and RRT-Star implementations.

Identifying victims was deemed to be a matter outside of this planner development scope, as it requires victim identification through computer vision implementation and a new planner. This can be developed as its own project.

The planner currently runs for a set number of iterations to develop the occupancy map. A desired amount of coverage can be set to terminate occupancy map generation instead.

# 6 Code

Installation of the UAV, Navigation, and Robotics toolboxes is required.

`integrated.mlx` demonstrates the complete implementation of generating random occupancy maps, executing the planner, and mapping the entire environment.

`MultiGoalRRTUAVSimulator.mlx` demonstrates purely the multi-goal RRT planner for a desired number of UAVs on a fixed map.